# Mars Habitat Automation System

A distributed IoT monitoring and control platform for Mars habitat simulation, built for the Advanced Programming Laboratory course.

# System Architecture Overview

1

### Data Ingestion

Sensor data collected from mars-simulator via REST and SSE endpoints

2

### Message Routing

Normalized messages distributed via RabbitMQ message broker

3

### Automation Engine

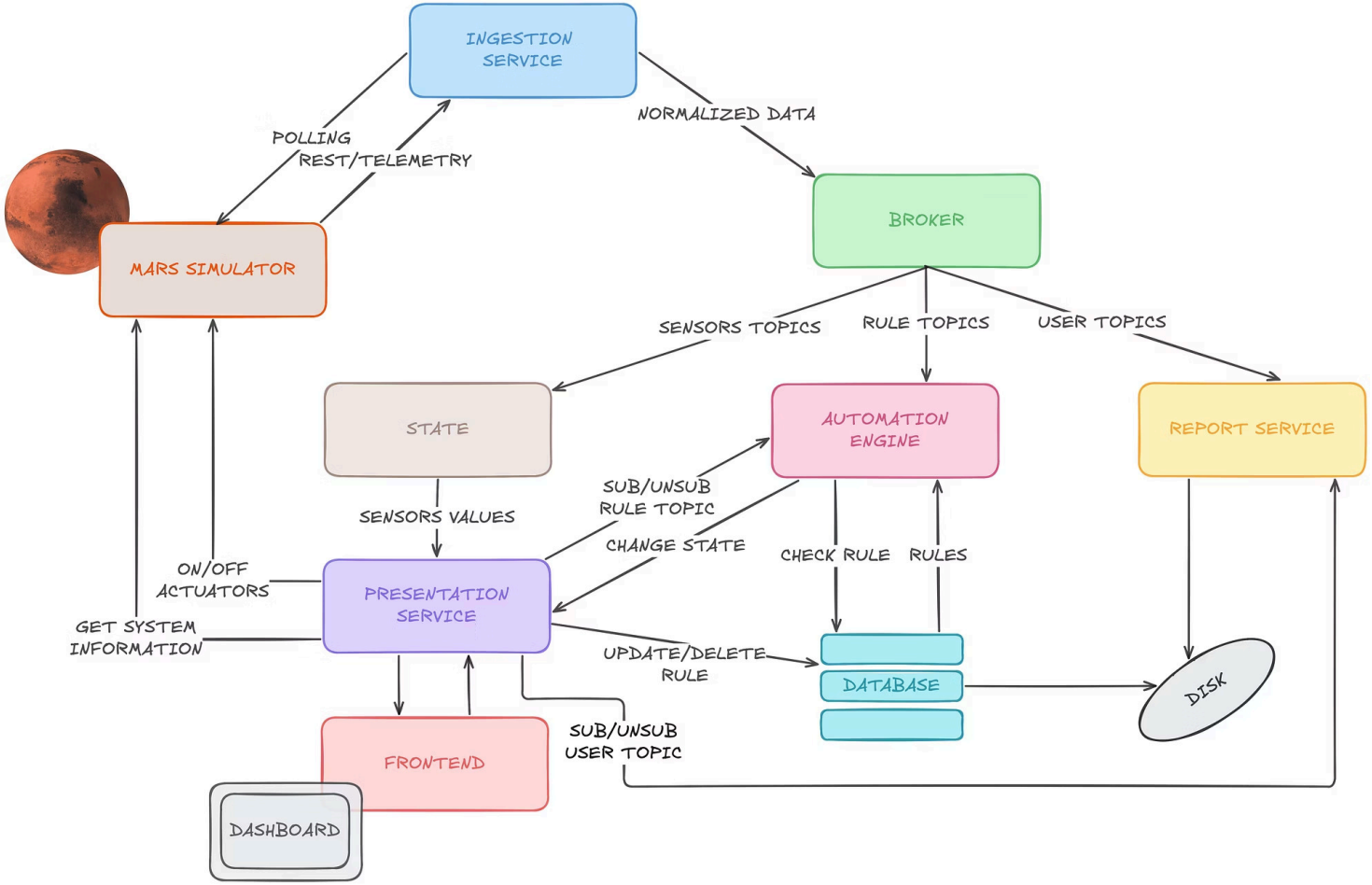
Rules applied to trigger actuator responses based on sensor conditions

4

### Dashboard UI

Real-time monitoring and manual control through web interface

# Architecture



# Technology Stack

## Message Broker

RabbitMQ with topic-based exchanges for real-time data distribution across services

## Data Persistence

PostgreSQL database storing automation rules with CRUD operations for rule management

## Core Services

Python 3.11 microservices handling ingestion, automation, state management, and reporting

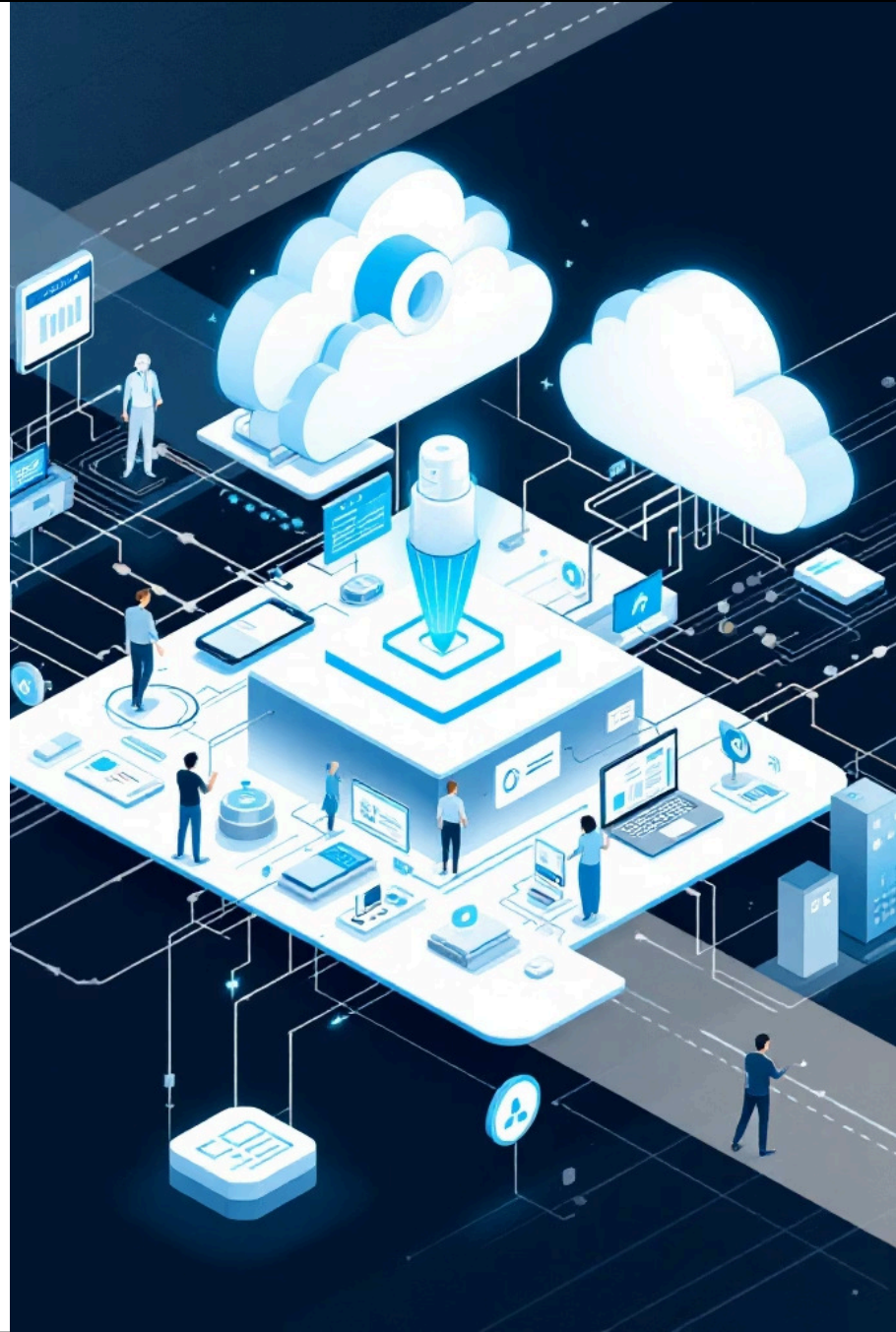
## Web Interface

Flask + Flask-SocketIO backend with HTML/JS frontend for real-time dashboard and controls

# Event Schema Normalisation

To handle incompatible device dialects, all incoming data is normalised into a unified internal format before being published to the message broker.

```
{
 "id": "string",
 "source": "rest | telemetry",
 "captured_at": "ISO_8601 timestamp",
 "status": "ok | warning",
 "measurements": [
 {
 "metric": "string",
 "value": "number",
 "unit": "string"
 }
 ],
 "topic": "string"
}
```



# Rule Model & Automation

### Dynamic Rules

IF-THEN rules evaluated upon arrival of new sensor events

### Database Persistence

Rules stored in PostgreSQL to survive service restarts

### Actuator Control

Automatic ON/OFF switching based on threshold conditions

# Monitoring User Stories

## View Sensors

Monitor available sensors with current values and measurement units

## Real-Time Telemetry

View live line charts for telemetry data including power and radiation

## Sensor Warnings

Visual alerts when sensors exceed critical thresholds or become unreachable

# Actuator Control Stories

## Manual Control

Operators can manually activate or deactivate actuators through the dashboard interface, providing override capability when automation needs to be bypassed.

## Automated Responses

System automatically triggers actuators when rule conditions are met, maintaining optimal environmental conditions without manual intervention.

**View actuator list and current states**

**Track activation history through system logs**

# Rule Management Stories

### Create Rules

Build new automation rules that react to sensor values and control environmental conditions

### Update Rules

Modify existing rules to respond to changing habitat conditions and requirements

### Delete Rules

Remove outdated automation logic that is no longer applicable to current operations

### Enable/Disable

Temporarily deactivate rules without deleting them for testing and troubleshooting

# System Features & Capabilities

### Rule Persistence

Automation logic preserved and restored automatically after system reboots or failures

### PDF Reports

Generate detailed sensor charts and statistics for archiving and analysis

### Action Logging

Comprehensive history of actuator state changes for auditing automation behaviour

# Key Achievements

**7**

**Microservices**

Containerised services working in harmony

**16**

**User Stories**

Complete feature coverage for operators

**99.9%**

**Real-Time**

WebSocket updates for live monitoring

This distributed automation platform demonstrates event-driven architecture principles, microservices communication patterns, and real-time system monitoring capabilities suitable for mission-critical environments.